

DAY 5 BACKEND TASK

MALINENI VAGDEVI

malinenv24@gmail.com

TASK 1: Create Product API Structure

STEP 1: Create project folder

Folder name: project-api

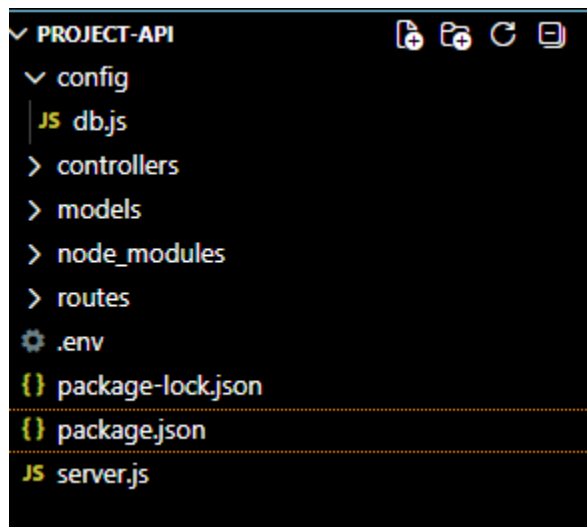
STEP 2: Initialize Node Project

`npm init -y`

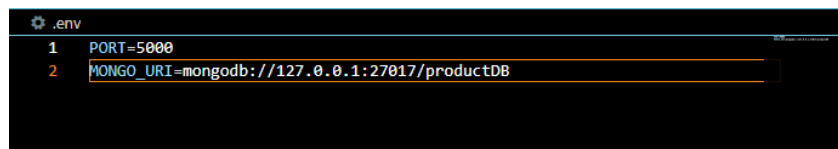
STEP 3: Install required packages

`npm install express mongoose dotenv`

STEP 4: Create Folders in VS Code



STEP 6 : Setup Environmental Variables



STEP 7: Configure MongoDB Connection

```
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4
5      try {
6
7          await mongoose.connect(process.env.MONGO_URI);
8
9          console.log('MongoDB Connected');
10
11      } catch (error) {
12
13          console.log(error);
14
15          process.exit(1);
16      }
17  };
18
19  module.exports = connectDB;
```

STEP 8: Setup Express Server

```
JS server.js > ...
1  const express = require('express');
2  const dotenv = require('dotenv');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6
7  const app = express();
8
9  app.use(express.json());
10
11  // Connect Database
12  connectDB();
13
```

```
// Test Route
app.get('/', (req, res) => {
    res.send('Product API Running');
});

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

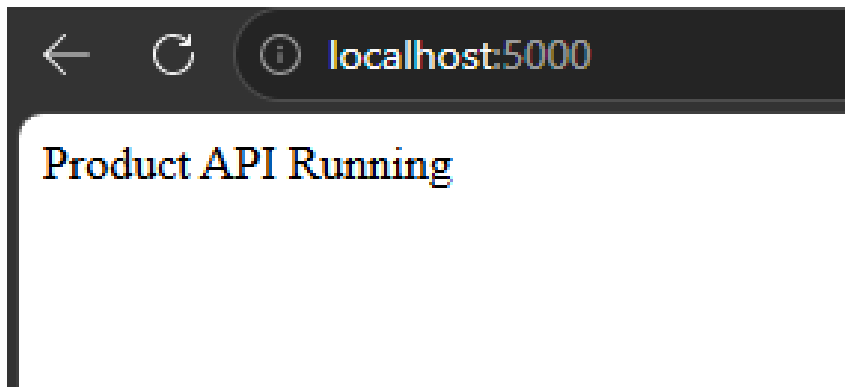
STEP 9: Run Server

```
PS C:\Users\V KRISHNA\project-api> npm run dev

> project-api@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: 📄 custom filepath { pat
Server running on port 5000
MongoDB Connected
█
```

STEP 10: Test Server



TASK 2 — Build Product Details API

will create:

Product Model, Product Controller, Product Routes, POST API

Step 1 — Create Product Model

```
models > JS Product.js > ...
1  const mongoose = require('mongoose');
2
3  const productSchema = new mongoose.Schema({
4    productName: {
5      type: String,
6      required: true
7    },
8
9    price: {
10     type: Number,
11     required: true
12   },
13
14   brand: {
15     type: String,
16     required: true
17   },
18
19   category: {
20     type: String,
21     required: true
22   },
23   stock: {
24     type: Number,
25     required: true
26   }
27 });
```

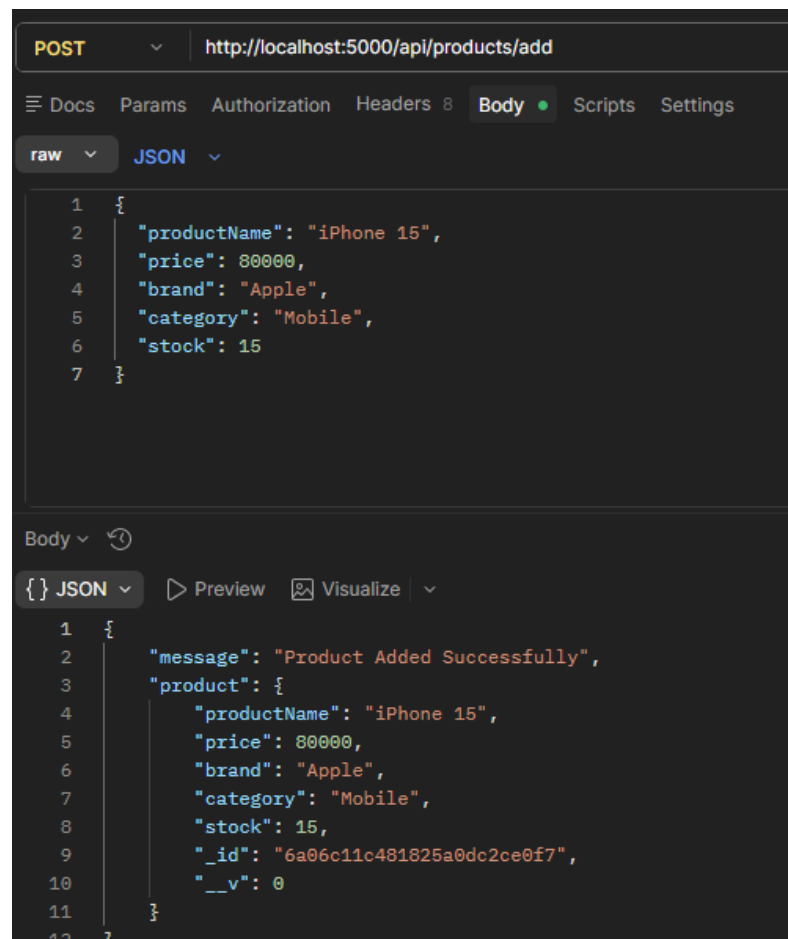
Step 2 — Create Product Controller

```
controllers > JS productController.js > ...
1  const Product = require('../models/Product');
2
3
4  // Add Product Controller
5  const addProduct = async (req, res) => {
6
7    try {
8
9      const {
10        productName,
11        price,
12        brand,
13        category,
14        stock
15      } = req.body;
16
17      const product = new Product({
18        productName,
19        price,
20        brand,
21        category,
22        stock
23      });
```

Step 3 — Create Product Routes

```
routes > JS productRoutes.js > ...  
1  const express = require('express');  
2  
3  const router = express.Router();  
4  
5  const {  
6    |   addProduct  
7  } = require('../controllers/productController');  
8  
9  
10 // POST API  
11 router.post('/add', addProduct);  
12  
13 module.exports = router;
```

Step 4 — Test API in Postman



The screenshot shows the Postman interface for a POST request to `http://localhost:5000/api/products/add`. The request body is a JSON object with the following fields:

```
{  
  "productName": "iPhone 15",  
  "price": 80000,  
  "brand": "Apple",  
  "category": "Mobile",  
  "stock": 15  
}
```

The response body is also shown in JSON format:

```
{  
  "message": "Product Added Successfully",  
  "product": {  
    "productName": "iPhone 15",  
    "price": 80000,  
    "brand": "Apple",  
    "category": "Mobile",  
    "stock": 15,  
    "_id": "6a06c11c481825a0dc2ce0f7",  
    "__v": 0  
  }  
}
```

Step 5 — Verify in MongoDB

```
PS C:\Users\V KRISHNA\project-api> mongosh
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverS
electionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      7.0.32
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-13T11:18:13.227+05:30: Access control is not enabled for the database
  . Read and write access to data and configuration is unrestricted
-----

test> use productDB
switched to db productDB
productDB> show collections
products
productDB> db.products.find()
[
  {
    _id: ObjectId('6a06c11c481825a0dc2ce0f7'),
    productName: 'iPhone 15',
    price: 80000,
    brand: 'Apple',
    category: 'Mobile',
    stock: 15,
    __v: 0
  }
]
productDB> 
```

We will see Product data stored in MongoDB

TASK 3 — Create Get Products API

This API will:

- fetch all products from MongoDB
- return product list

Step 1 — Open Product Controller

Step 2 — Add Get Products Controller

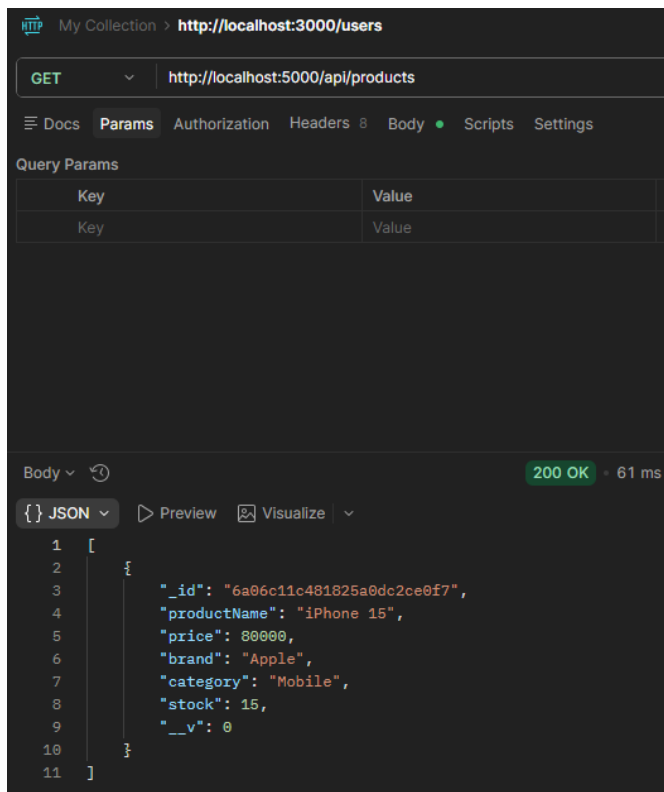
Step 3 — Update Exports

```
41
42 // Get All Products
43 const getProducts = async (req, res) => {
44
45   try {
46
47     const products = await Product.find();
48
49     res.json(products);
50
51   } catch (error) {
52
53     res.status(500).json({
54       |   message: 'Server Error'
55     });
56   }
57 };
58
59
60 module.exports = {
61   addProduct,
62   getProducts
63 };
```

Step 4 — Update Routes

```
routes > JS productRoutes.js > ...
1  const express = require('express');
2
3  const router = express.Router();
4
5  const {
6    addProduct,
7    getProducts
8  } = require('../controllers/productController');
9
10
11 // POST API
12 router.post('/add', addProduct);
13
14
15 // GET API
16 router.get('/', getProducts);
17
18 module.exports = router;
```

Step 5 — Test GET API in Postman



The screenshot shows the Postman interface with the following details:

- URL:** `http://localhost:5000/api/products`
- Method:** `GET`
- Params:** No query parameters are present.
- Status:** `200 OK` (61 ms)
- Body:** JSON format showing an array of one product object.

```
1  [
2    {
3      "_id": "6a06c11c481825a0dc2ce0f7",
4      "productName": "iPhone 15",
5      "price": 80000,
6      "brand": "Apple",
7      "category": "Mobile",
8      "stock": 15,
9      "__v": 0
10   }
11 ]
```

Your GET Products API is now working successfully

GITHUB REPOSITORY URL

<https://github.com/vagdevi-edu/project-api>